11

Transport Equations in Metals, Organic Materials, and Disordered Media

This chapter describes transport equations in materials other than low-defect, inorganic semiconductors.

- Singlet Exciton Equation
- Transport in Metals
- Conductive Insulators

Singlet Exciton Equation

In organic semiconductor devices, besides the simulation of the electrical part consisting of solving the Poisson and continuity equations, the *singlet exciton equation* is introduced to account for optical properties of these materials related to Frenkel excitons.

This equation takes into account only singlet excitons because triplet excitons do not contribute directly to light emission. The equation models the dynamics of the generation, diffusion, recombination, and radiative decay of singlet excitons in organic semiconductors.

The singlet exciton equation, governing the transport of excitons in organic semiconductors, is given by:

Equation 143

$$\frac{\partial n_{\mathrm{se}}}{\partial t} = R_{\mathrm{bimolec}} + \nabla \cdot D_{\mathrm{se}} \nabla n_{\mathrm{se}} - \frac{n_{\mathrm{se}} - n_{\mathrm{se}}^{\mathrm{eq}}}{\tau} - \frac{n_{\mathrm{se}} - n_{\mathrm{se}}^{\mathrm{eq}}}{\tau_{\mathrm{trap}}} - R_{\mathrm{se}}$$

where:

- n_{se} is the singlet exciton density.
- R_{bimolec} is the carrier bimolecular recombination rate acting as a singlet exciton generation term.
- D_{se} is the singlet exciton diffusion constant.

- τ, τ_{trap} are the singlet exciton lifetimes.
- R_{se} is the net singlet exciton recombination rate.

The first term on the right-hand side of Equation 143 accounts for the creation of excitons through bimolecular recombination (see Bimolecular Recombination Model on page 602). The second term describes exciton diffusion; while radiative decay associated with light emission is accounted for by the third and fourth terms.

Nonradiative exciton destruction and conversion of excitons to free electron and free hole populations are described by the net exciton recombination rate (fifth term):

Equation 144

$$\begin{aligned} R_{\mathrm{se}} &= R_{\mathrm{se}-nf} + R_{\mathrm{se}-pf} \\ R_{\mathrm{se}-nf} &= (v_{\mathrm{se}} + v_n)\,\sigma_{\mathrm{se}-n} n_{\mathrm{se}} n \\ R_{\mathrm{se}-pf} &= (v_{\mathrm{se}} + v_p)\,\sigma_{\mathrm{se}-p} n_{\mathrm{se}} p \end{aligned}$$

where:

- $R_{\mathrm{se}-nf}$, $R_{\mathrm{se}-pf}$ are the exciton recombination rate due to free electron and free hole populations, respectively.
- $v_{\mathrm{se}} = v_{\mathrm{se},0}$ is the exciton velocity.
- $v_n = v_{n,0}\sqrt{T / 300\mathrm{K}}$ and $v_p = v_{p,0}\sqrt{T / 300\mathrm{K}}$ are the electron and hole velocities. $v_{\mathrm{se},0}$, $v_{n,0}$, and $v_{p,0}$ are set in the parameter file.
- $\sigma_{\mathrm{se}-n}$ and $\sigma_{\mathrm{se}-p}$ are the reaction cross sections between the singlet exciton, and the electron and hole, respectively.

In addition, the net exciton recombination rate R_{se} might contain exciton interface dissociation rates converted to volume terms if the exciton interface dissociation model is activated (see Exciton Dissociation Model on page 603).

Boundary and Continuity Conditions for Singlet Exciton Equation

At electrodes and thermodes, equilibrium is assumed for the singlet exciton population:

Equation 145

$$n_{\mathrm{se}}(T) = n_{\mathrm{se}}^{\mathrm{eq}}(T) = \gamma\, g_{\mathrm{ex}}(N_{\mathrm{C}}(T) + N_{\mathrm{V}}(T))\, \exp\left(-\frac{E_{\mathrm{g,eff}} - E_{\mathrm{ex}}}{kT}\right)$$

where γ = 1/4, and g_{ex} and E_{ex} are the singlet exciton degeneracy factor and binding energy, respectively.

Conditions for the singlet exciton equation at interfaces depend on the interface type. The interface type is dictated by the two regions adjacent to the interface.

For interfaces where the singlet exciton equation is solved only in one of the regions of the interface, the boundary conditions imposed are either Equation 145 (default) or $\nabla n_{se} \cdot \hat{n} = 0$ (zero flux), depending on the user selection.

For a heterointerface where the singlet exciton equation is solved in both regions, the continuity conditions imposed are thermionic-like. In this case, you can select the barrier ΔE to be the difference between the band gaps, the conduction bands, or the valence bands of the two regions.

Assuming that at heterointerfaces between materials 1 and 2 (that is, regions 1 and 2), $\Delta E > 0$ ($E_{g,2} > E_{g,1}$ for example), and $j_{se,2}$ is the singlet exciton flux leaving material 2 and $j_{se,1}$ is the singlet exciton flux entering material 1, the interface boundary condition can be written as:

Equation 146

$$j_{se,1} = j_{se,2}$$

$$j_{se,2} = v_{si}\left(n_{se,2} - n_{se,1}\exp\left(-\frac{\Delta E}{kT}\right)\right)$$

where $n_{se,2}$ and $n_{se,1}$ are the singlet exciton densities of materials 2 and 1 at the heterointerface, and v_{si} is the organic heterointerface emission velocity defined in the parameter file.

Using the Singlet Exciton Equation

To activate the singlet exciton equation, the keyword `SingletExciton` must be specified in the `Coupled` statement of the `Solve` section (see Coupled Statement on page 214). Then, the regions where the equation will be solved are selected by specifying the keyword `SingletExciton` with different options in the `Physics` section of the respective regions (by default, none of the regions is selected for solving the singlet exciton equation, so you must select at least one region).

The singlet exciton generation and recombination terms in Equation 143 are switched off by default. They can be activated regionwise by specifying the corresponding keyword as an option for `Recombination` in the `SingletExciton` section of the respective region `Physics` section (see Table 271 on page 1797).

The keyword `Bimolecular` activates the bimolecular generation (first term). The keywords `Radiative` and `TrappedRadiative` switch on radiative decay of singlet excitons either

directly or trap-assisted (third and fourth terms). Nonradiative exciton destruction (fifth term) is activated by `eQuenching` and `hQuenching`. For example:

```
Physics(Region="EML-ETL") {
   SingletExciton(Recombination(Bimolecular eQuenching hQuenching))
}
```

For interfaces where the singlet exciton equation is solved only in one of the regions of the interface, the default boundary conditions are defined by Equation 145.

To switch to zero flux boundary condition $\nabla n_{se} \cdot \hat{n} = 0$, the keyword `FluxBC` must be specified as an option for `SingletExciton` in the interface `Physics` section:

```
Physics(RegionInterface="Region_0/Region_2") {
   SingletExciton(FluxBC)
}
```

For heterointerfaces, the default barrier type for the singlet exciton flux injection across the interface (defined in Equation 146) is the bandgap energy difference.

The barrier can be switched to conduction band or valence band type by specifying the keyword `BarrierType` with the option `CondBand` or `ValBand` in the `SingletExciton` section of the heterointerface `Physics` section:

```
Physics(RegionInterface="Region_0/Region_2") {
   SingletExciton(BarrierType(CondBand)
}
```

Parameters used with the singlet exciton equation must be specified in the `SingletExciton` section of the parameter file.

Table 36 Parameters for singlet exciton equation

Symbol	Parameter name	Default	Location	Unit
γ	gamma	0.25	region	1
τ	tau	1×10^{-7}	region	s
τ_{trap}	tau_trap	1×10^{-8}	region	s
L_{diff}	l_diff	1×10^{-3}	region	cm
$D_{se} = L_{diff}^2 / \tau$	–	–	region	cm^2/s
σ_{se-n}	ex_cXsection	1×10^{-8}	region	cm^2
σ_{se-p}	ex_cXsection	1×10^{-8}	region	cm^2
E_{ex}	Eex	0.015	region	eV

Table 36 Parameters for singlet exciton equation (Continued)

Symbol	Parameter name	Default	Location	Unit
g_{ex}	gex	4	region	1
$v_{se,0}$	vth	1×10^7	region	cm/s
$v_{n,0}$	vth_car	1×10^3	region	cm/s
$v_{p,0}$	vth_car	1×10^3	region	cm/s
v_{si}	vel	1×10^8	interface	cm/s

By default, Sentaurus Device uses built-in models to compute material resistivity. However, material resistivity can be defined by using the `SFactor` keyword as `SFactor= "ElectricalResistivity"`. The `SFactor` keyword must be specified in the `Physics` section of the command file as follows:

```
Physics { MetalResistivity (SFactor="ElectricalResistivity") }
```

With this keyword, the metal resistivity (ρ) can be read from the file specified by `DevFields` in the `File` section as:

```
File {
   DevFields = "ElectricalResistivity.tdr"
}
```

Transport in Metals

The simulation of current transport in metals or semi-metals is important for interconnection problems in ICs. The current density in metals is given by:

Equation 147

$$\vec{J}_M = -\sigma(\nabla\Phi_M + P\nabla T)$$

where:

- σ is the metal conductivity.
- Φ_M is the Fermi potential in the metal.
- P is the metal thermoelectric power.
- T is the lattice temperature.

The second term in Equation 147 accounts for the Seebeck effect, and it is nonzero only when computation of metal thermoelectric power is activated (see Thermoelectric Power on page 1146). For the steady-state case, $\nabla \cdot \vec{J}_{\mathrm{M}} = 0$. Therefore, the equation for the Fermi potential inside of metals is:

Equation 148

$$\nabla \cdot (\sigma (\nabla \Phi_{\mathrm{M}} + P \nabla T)) = 0$$

The following temperature dependence is applied for ρ = 1/σ:

Equation 149

$$\rho = \rho_0 \left(1 + \alpha_1 (T - 273\,\mathrm{K}) + \alpha_2 (T - 273\,\mathrm{K})^2 \right)$$

All these resistivity parameters can be specified in the parameter file as:

```
Resistivity {
* Resist(T) = Resist0*( 1 + TempCoef*(T-273) + TempCoef2*(T-273)^2 )
  Resist0   = <value>     # [Ohm*cm]
  TempCoef  = <value>     # [1/K]
  TempCoef2 = <value>     # [1/K^2]
}
```

No specific keyword is required in the command file because Sentaurus Device recognizes all conductor regions and applies the appropriate equations to these regions and interfaces. The metal conductivity equation Equation 148 is a part of the `Contact` equation, which is solved automatically by default. If the keyword `NoAutomaticCircuitContact` is specified in the `Math` section or only the Poisson equation is solved, the `Contact` equation must be added explicitly in the `Coupled` statement to account for conductivity in metals.

To switch off current transport in metals, specify the keyword `-MetalConductivity` in the `Math` section.

Note:

If you use the `-MetalConductivity` option, then all contacts placed directly on metal regions will be electrically disconnected from these regions.

Electric Boundary Conditions for Metals

The following boundary conditions are used:

- At contacts connected to metal regions, the Dirichlet condition $\Phi_{\mathrm{M}} = V_{\mathrm{applied}}$ is applied for the Fermi potential.
- Interface conditions always include the displacement current $\vec{J}_{\mathrm{D}}$ to ensure conservation of current.

- For interfaces between metal and insulator, the equations are:

Equation 150

$$\vec{J}_{\mathrm{M}} \cdot \hat{n} = \vec{J}_{\mathrm{D}} \cdot \hat{n}$$
$$\phi = \Phi_{\mathrm{M}} - \Phi_{\mathrm{MS}}$$

where Φ_{MS} is the workfunction difference between the metal and an intrinsic semiconductor selected (internally by Sentaurus Device) as a reference material, and $\hat{n}$ is the unit normal vector to the interface. The electrostatic potential inside metals is computed as $\phi = \Phi_{\mathrm{M}} - \Phi_{\mathrm{MS}}$.

- At metal–semiconductor interfaces, by default, there is an Ohmic boundary condition. Schottky boundary conditions can be selected as an option.

The Ohmic boundary condition at metal–semiconductor interfaces reads:

Equation 151

$$\vec{J}_{\mathrm{M}} \cdot \hat{n} = \left(\vec{J}_n + \vec{J}_p + \vec{J}_{\mathrm{D}}\right) \cdot \hat{n}$$
$$\phi = \Phi_{\mathrm{M}} + \phi_0$$
$$n = n_0$$
$$p = p_0$$

where:

- ϕ_0 is the equilibrium electrostatic potential (the built-in potential).
- n_0 and p_0 are the electron and hole equilibrium concentrations (see Ohmic Contacts on page 289).

There are several options for the Schottky interface (refer to the equations in Schottky Contacts on page 292). The potential barrier between metal and semiconductor is computed automatically, and the barrier tunneling model (see Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 916) and the barrier lowering model (see Barrier Lowering at Schottky Contacts on page 296) can be applied.

To select these models, use the following syntax in the command file:

```
Physics(MaterialInterface = "Metal/Silicon") { Schottky
   eRecVelocity=1e6 hRecVelocity=1e6 }
Physics(MaterialInterface = "Metal/Silicon") { Schottky
   BarrierLowering }
Physics(MaterialInterface = "Metal/Silicon") { Schottky }
```

The default values of electron and hole surface recombination velocities are `eRecVel=2.573e6` and `hRecVel=1.93e6`, respectively. Similarly to `Electrode` conditions, if `Schottky` is not specified but the `SurfaceSRH` keyword is specified, then Sentaurus

Device will apply Ohmic boundary conditions to the electrostatic potential and the surface recombination for both electron and hole carrier current densities on such interfaces.

Note:

At nonmetal interfaces, the keyword `SurfaceSRH` activates a different model from the one described in this section (see Surface SRH Recombination Model on page 540).

Both Ohmic and Schottky interfaces support a distributed resistance model similar to the one described in Resistive Contacts on page 302. If a distributed resistance is specified at the interface, a distributed voltage drop $\Delta\phi\ (R_{\mathrm{d}}) = R_{\mathrm{d}}\left(\vec{J}_n + \vec{J}_p + \vec{J}_{\mathrm{D}}\right) \cdot \hat{n}$ is applied to the existing boundary condition for potential, where $\Delta\phi\ (R_{\mathrm{d}})$ is the voltage drop across the interface, and R_{d} is the distributed resistance at the interface. In addition, for Ohmic interfaces, emulation of a Schottky interface using a doping-dependent resistivity model is possible. These options are activated as following:

```
# Distributed resistance at Ohmic interface
Physics(MaterialInterface = "Metal/Silicon") {DistResist=1e-6}

# Distributed resistance at Schottky interface
Physics(MaterialInterface = "Metal/Silicon") {Schottky DistResist=1e-6}

# Distributed resistance at Ohmic interface emulating
# a Schottky interface
Physics(MaterialInterface = "Metal/Silicon") {DistResist=SchottkyResist}
```

For Schottky interfaces, the distributed resistance model works with all other options previously described.

For all other metal boundaries, the Neumann condition $\vec{J}_{\mathrm{M}} \cdot \hat{n} = 0$ is applied.

Note:

By default, Sentaurus Device computes output currents through a contact using integration over the associated wells. For metal contacts, the integration can be switched off and the local current can be used instead by specifying the keyword `DirectCurrent` in the `Math` section. This option might produce a wrong current at the metal contact.

Metal Workfunction

To specify the metal workfunction, use the `Bandgap` parameter set. For example:

```
Material = "Gold" {
   Bandgap { WorkFunction = 5 # [eV] }
}
```

It is also possible to account for workfunction variations within the metal.

If such variations can be characterized by mole-fraction variations and if the metal is treated as a mole fraction–dependent material (see Mole-Fraction Materials on page 67), then `WorkFunction` in the `Bandgap` parameter set can be specified as a mole fraction–dependent quantity. For example:

```
Material = "Metal" {
   Bandgap {
      Xmax(0)=0.0
      Xmax(1)=0.2
      WorkFunction(0) = 4.53# [eV]
      WorkFunction(1) = 4.38# [eV]
   }
}
```

The positional dependency of the workfunction in metals also can be specified directly in the `Physics` section of the command file using one of three different methods:

```
Physics (Material = "<name>" | Region = "<name>") {
   MetalWorkfunction (
      [ Workfunction=<wf> ] |
      [ Workfunction=(<wf1>, <x1>, <y1>, <z1>)
      Workfunction=(<wf2>, <x2>, <y2>, <z2>)
      ...
      Workfunction=(<wfN>, <xN>, <yN>, <zN>) ] |
      [ SFactor= "<dataset_name-or-pmi_model_name>" [Factor=<scale>]
        [Offset=<offset>]]
   )
}
```

The first method (`Workfunction=<wf>`) simply specifies a constant workfunction value for the material or region that overrides any specification from the parameter file.

The next method allows the specification of an arbitrary number of workfunction–position quadruplets. The workfunction at a vertex in the metal is assigned the value of the workfunction from the quadruplet whose position is closest to the vertex.

Alternatively, you can use the `SFactor` parameter to specify a dataset name (which includes the PMI user fields `PMIUserField0` through `PMIUserField299`) from which the vertex workfunction values will be taken, or a space factor PMI model can be specified that calculates the vertex workfunction values directly (see Space Factor on page 1624). If `Factor=<scale>` is specified, then the `SFactor` values are normalized and then are multiplied by this factor. If `Offset=<offset>` is specified, then this value is added to the raw or scaled `SFactor` values.

Metal Workfunction Randomization

The workfunction in metal regions can be randomized using specifications in the command file:

```
Physics (Material = "<name>" | Region = "<name>") {
   MetalWorkfunction (
```

```
        Randomize (
           AverageGrainSize=<size>                # [micrometers]
           GrainProbability=(<P1>, <P2>, ...)
           GrainWorkfunction=(<wf1>, <wf2>, ...)  # [eV]
           RandomSeed=<seed>
           AtInsulatorInterface
           UniformDistribution
        )
    )
 }
```

The approach taken assumes that the metal consists of randomized grains of varying size and shape that can be characterized with an average grain size. It also assumes that the grains in the metal occur with a finite number of orientations, and the number of grains for each orientation can be characterized with a probability of occurrence. All grains of the same orientation are assumed to have the same workfunction, but the workfunction can be different for each orientation.

`AverageGrainSize` is used to determine the average number of grains for a region, which then is used as the expectation value for a Poisson distribution, random number generator to determine the actual number of grains for the region. Each grain can have a different shape and size.

You can specify an arbitrary number of `GrainProbability` values, but their sum must be equal to one. In addition, there must be a one-to-one correspondence between `GrainProbability` values and `GrainWorkfunction` values.

By default, the seed for the random number generator used in the randomization process is different for every simulation. However, `RandomSeed` can be specified if required. This allows a particular randomization to be repeated if the same seed is used in a subsequent simulation of the same device on the same computer.

The `AtInsulatorInterface` option confines the randomization to the metal–insulator interface vertices. However, the resulting workfunction values at the interface will be exactly the same as those obtained when the entire metal region is randomized.

The `UniformDistribution` option *attempts* to use grains with a uniform size that are distributed uniformly. The success of this is highly dependent on the grid used in the metal region.

The workfunction in metal regions can be saved in a plot file for visualization by specifying `MetalWorkfunction` in the `Plot` section of the command file.

Temperature in Metals

In metals, only the lattice temperature is defined. If the lattice temperature is solved, in metals, it obeys the following equation:

Equation 152

$$c_L \frac{\partial T}{\partial t} - \nabla \cdot \kappa \nabla T = -\nabla \psi_M \cdot \vec{J}_M$$

where κ is the thermal conductivity (see Thermal Conductivity on page 1131). If current transport in metals is switched off, then the right-hand side of Equation 152 vanishes. However, a heat flow in metals can still be simulated.

Conductive Insulators

Leakage currents in dielectrics can be modeled by allowing dielectrics to have conductive properties. The conductive insulator (leaky insulator) model implements dielectrics that have nonzero (but typically low) conductivity (a metal-like property), besides the pure dielectric properties.

In a conductive insulator, the electrostatic potential is computed using the Poisson equation, similar to a pure dielectric. As in metals, conductivity is modeled by the Fermi potential (solving Equation 148) and then the leakage current is computed using Equation 147. Since the model does not allow net charge in the conductive insulator, the Poisson equation remains unchanged by adding conductive properties to the insulator.

The equation for the Fermi potential inside a conductive insulator is the same used for metals (see Equation 148), where σ is now the conductivity of the conductive insulator and $\Phi_M = \Phi_{CI}$ is the Fermi potential in the conductive insulator.

The following boundary conditions are used for this equation:

- At contacts connected to conductive insulator regions, the Dirichlet condition $\Phi_{CI} = V_{applied}$ is applied for the Fermi potential.
- Interface conditions always include the displacement current $\vec{J}_D$ in insulators, conductive insulators, and semiconductors to ensure conservation of current.
- For conductive insulator–semiconductor interfaces, Ohmic-like boundary conditions, thermionic-like boundary conditions, and boundary conditions for floating regions are available.

Leakage from or to a semiconductor to or from a conductive insulator is mainly determined by two factors: the conductive properties of the interface and the bulk resistivity of the conductive insulator. The bulk resistivity-limited conduction corresponds to the case when

the current flow is limited by the high resistivity of the conductive insulator region, and the interface has zero resistivity.

This case is modeled by the Ohmic-like boundary conditions:

Equation 153

$$\vec{J}_{\mathrm{CI}} \cdot \hat{n} = \left(\alpha \vec{J}_n + (1 - \alpha) \vec{J}_p \right) \cdot \hat{n}$$
$$\Phi_{\mathrm{CI}} = \phi - \phi_0$$

where:

- Φ_{CI} and $\vec{J}_{\mathrm{CI}}$ are the Fermi potential and current density on the interface.
- ϕ is the electrostatic potential (the solution of the Poisson equation in insulator and semiconductor regions).
- ϕ_0 is the built-in potential.
- $\hat{n}$ is the unit normal vector to the interface.
- $\vec{J}_n$ and $\vec{J}_p$ are the electron and hole currents on the semiconductor side of the interface.
- α is the fraction (0 ≤ α ≤ 1) of the current density of the conductive insulator going to the electron current density.

Thermionic-like emission at the interface models the case where conduction is limited by both interface properties and the bulk resistivity of the conductive insulator.

Thermionic boundary conditions read:

Equation 154

$$\vec{J}_{\mathrm{CI}} \cdot \hat{n} = \left(\vec{J}_{n,\mathrm{th}} + \vec{J}_{p,\mathrm{th}} \right) \cdot \hat{n}$$
$$\vec{J}_{n,\mathrm{th}} = aq \left[v_n(T_n) n - v_n(T) \left(\frac{T}{T_n} \right)^{1.5} N_{\mathrm{C}} \exp\left(\frac{E_{\mathrm{F}}^{\mathrm{CI}} - E_{\mathrm{C}}}{kT} \right) \right]$$
$$\vec{J}_{p,\mathrm{th}} = aq \left[v_p(T_p) p - v_p(T) \left(\frac{T}{T_p} \right)^{1.5} N_{\mathrm{V}} \exp\left(\frac{E_{\mathrm{V}} - E_{\mathrm{F}}^{\mathrm{CI}}}{kT} \right) \right]$$

where:

- $\vec{J}_{n,\mathrm{th}}$ and $\vec{J}_{p,\mathrm{th}}$ are the electron and hole thermionic-emission current densities on the interface from the semiconductor side.
- $v_n(T) = (kT / 2\pi m_n)^{0.5}$ and $v_p(T) = (kT / 2\pi m_p)^{0.5}$ are 'emission velocities' at the interface.
- T, T_n, and T_p are the lattice, electron, and hole temperatures, respectively.

- E_F^{CI}, E_C, and E_V are the conductive insulator Fermi-level energy, conduction band energy, and valence band energy, respectively.
- a is a constant set through the parameter file, which has the default value of 2.

By using a conductive insulator layer between a semiconductor floating region and a semiconductor region, leakage from a semiconductor floating-gate can be emulated. The boundary condition used in this case is:

Equation 155

$$\vec{J}_{CI} \cdot \hat{n} = -\sigma \nabla \Phi$$

where Φ is the Fermi potential close to the interface. In this case, during transient simulations, the charge update on the floating gate is computed as $J_{CI}\Delta t$, where J_{CI} is the total current at the conductive insulator–floating gate interface, and Δt is the current time step. The activation of this type of boundary condition is automatic if the boundary condition at the conductive insulator–semiconductor floating-gate interface is Ohmic and the simulation is transient (to allow for charging and discharging of the floating gate).

The model is activated by using the keyword `CondInsulator` in the `Physics` section to make the dielectric conductive and by adding the `CondInsulator` equation in the `Solve` section to actually solve the Fermi potential:

```
Physics(Region="insulator"){
   ...
   CondInsulator
   ...
}

Solve {
   ...
   Coupled {Poisson Electron Hole Contact CondInsulator}
   ...
}
```

The parameters for the resistivity of conductive insulators (see Equation 149) are specified in the `Resistivity` parameter set:

```
Material = "Si3N4" {
   Resistivity {
      * Resist(T) = Resist0 * (1 + TempCoef * (T-273))
      Resist0 = 3.0e9                                  # [Ohm*cm]
      TempCoef = 43.0e-4                               # [1/K]
   }
}
```

Boundary conditions according to Equation 153 are used by default. The parameter α in Equation 153 is given by the value of `curw` in the `CondInsCurr` parameter set:

```
MaterialInterface = "Si3N4/AlGaN" {
   CondInsCurr {
      curw = 1.0   # [1]
   }
}
```

Thermionic emission at the semiconductor–conductive insulator interface is activated by adding `eThermionic` or `hThermionic` in the `Physics` section:

```
Physics(MaterialInterface="Si3N4/AlGaN") {
   *----- Insulator/AlGaN -----
   ...
   eThermionic
   hThermionic
   ...
}
```

where the parameter a (see Equation 154) is specified in the parameter file:

```
MaterialInterface = "Si3N4/AlGaN" {
   ThermionicEmission {
      A = 2, 2
   }
}
```

For nonthermionic interfaces, an equilibrium solution is needed internally to solve the `CondInsulator` equation. The equilibrium solution is computed automatically by a nonlinear iteration. Sentaurus Device allows you to specify numeric parameters of the nonlinear iteration as options to the keyword `EquilibriumSolution` in the `Math` section (see Equilibrium Solution on page 257 and Table 224 on page 1750).

The following example forces the Newton solver to solve up to 100 iterations:

```
Math {
   ...
   EquilibriumSolution(Iterations=100)
   ...
}
```